PROGRAMMING IN JAVA

ASSIGNMENT 1 – CAMPUS SMART-PARKING SYSTEM

Design, Implement and Validate an Object-Oriented Model

Student Name: D DHANUJA

Register Number: 192311427

Section / Batch: 2023

Submission Date: 17/10/2026

GitHub Repository Link:
https://github.com/DhanujaDhanasekar666/CSA0902-192311427

CODE:

```
import java.time.Duration;
import java.time.LocalDateTime;
import java.util.*;

/**
 * Campus Smart-Parking System
 * Demonstrates encapsulation, inheritance, overriding, polymorphism,
 * abstraction, interfaces, packages-ready class design, validation and tests.
 *
 * Compile: javac CampusSmartParkingSystem.java
 * Run:     java CampusSmartParkingSystem
```

```
 */
interface Chargeable {
    double chargingFee(int kWh);
}

interface PricingRule {
    double calculate(Vehicle vehicle, long minutes, boolean
monthlyPass);
}

abstract class Vehicle {
    private final String registrationNumber;

    protected Vehicle(String registrationNumber) {
        if (registrationNumber == null ||
registrationNumber.trim().isEmpty()) {
            throw new IllegalArgumentException("Registration number
cannot be empty.");
        }
        this.registrationNumber =
registrationNumber.trim().toUpperCase(Locale.ROOT);
    }

    public final String getRegistrationNumber() {
        return registrationNumber;
    }
```

```
    public abstract double ratePerHour();

    @Override
    public final boolean equals(Object obj) {
        if (this == obj) return true;
        if (!(obj instanceof Vehicle)) return false;
        Vehicle other = (Vehicle) obj;
        return registrationNumber.equals(other.registrationNumber);
    }

    @Override
    public final int hashCode() {
        return registrationNumber.hashCode();
    }

    @Override
    public String toString() {
        return getClass().getSimpleName() + "(" + registrationNumber +
")";
    }
}

class TwoWheeler extends Vehicle {
```

```
    public TwoWheeler(String registrationNumber) {
super(registrationNumber); }
    @Override public double ratePerHour() { return 10.0; }
}

class Car extends Vehicle {
    public Car(String registrationNumber) {
super(registrationNumber); }
    @Override public double ratePerHour() { return 30.0; }
}

class ElectricCar extends Car implements Chargeable {
    public ElectricCar(String registrationNumber) {
super(registrationNumber); }
    @Override public double ratePerHour() { return 30.0; }
    @Override public double chargingFee(int kWh) {
        if (kWh < 0) throw new IllegalArgumentException("kWh cannot
be negative.");
        return kWh * 12.0;
    }
}

class DeliveryVan extends Vehicle {
    private static final double PEAK_MULTIPLIER = 2.0;
    public DeliveryVan(String registrationNumber) {
super(registrationNumber); }
```

```
    @Override public double ratePerHour() { return 60.0; }
    public double rateForEntryTime(LocalDateTime entry) {
        int hour = entry.getHour();
        return (hour >= 9 && hour < 17) ? ratePerHour() *
PEAK_MULTIPLIER : ratePerHour();
    }
}

class EBike extends Vehicle {
    public EBike(String registrationNumber) {
super(registrationNumber); }
    @Override public double ratePerHour() { return 8.0; }
}

/** Default policy: each started hour is charged as a full hour. */
class StandardPricingRule implements PricingRule {
    @Override
    public double calculate(Vehicle vehicle, long minutes, boolean
monthlyPass) {
        if (minutes < 0) throw new IllegalArgumentException("Duration
cannot be negative.");
        if (monthlyPass) return 0.0;
        long billableHours = (minutes + 59) / 60; // ceiling division,
integer arithmetic
        double rate = vehicle.ratePerHour();
        if (vehicle instanceof DeliveryVan) {
```

```
            // The assignment specifies double car rate from 9 AM to 5
PM.
            // Entry-time handling is performed by the facility at checkout.
            rate = vehicle.ratePerHour();
        }
        return billableHours * rate;
    }
}

class ParkingTicket {
    private static long nextTicketNumber = 1000;
    private final String ticketNumber;
    private final Vehicle vehicle;
    private final int floor;
    private final int slot;
    private final LocalDateTime entryTime;
    private final boolean monthlyPass;

    public ParkingTicket(Vehicle vehicle, int floor, int slot,
                         LocalDateTime entryTime, boolean monthlyPass) {
        this.ticketNumber = "T" + (++nextTicketNumber);
        this.vehicle = Objects.requireNonNull(vehicle, "vehicle");
        if (floor < 0 || slot < 0) throw new
IllegalArgumentException("Invalid slot.");
        this.floor = floor;
```

```
        this.slot = slot;
        this.entryTime = Objects.requireNonNull(entryTime,
"entryTime");
        this.monthlyPass = monthlyPass;
    }

    public String getTicketNumber() { return ticketNumber; }
    public Vehicle getVehicle() { return vehicle; }
    public int getFloor() { return floor; }
    public int getSlot() { return slot; }
    public LocalDateTime getEntryTime() { return entryTime; }
    public boolean hasMonthlyPass() { return monthlyPass; }
}

class ParkingFacility {
    // A jagged array allows each floor to have a different number of
slots.
    private final String[][] slots;
    private final Map<String, ParkingTicket> activeTickets = new
LinkedHashMap<>();
    private final Map<String, ParkingTicket> activeByRegistration =
new HashMap<>();
    private final PricingRule pricingRule;
    private static int totalTicketsIssued = 0;

    public ParkingFacility(int[] slotsPerFloor) {
```

```
        this(slotsPerFloor, new StandardPricingRule());
    }

    public ParkingFacility(int[] slotsPerFloor, PricingRule pricingRule)
{
        if (slotsPerFloor == null || slotsPerFloor.length == 0)
            throw new IllegalArgumentException("At least one floor is
required.");
        this.slots = new String[slotsPerFloor.length][];
        for (int f = 0; f < slotsPerFloor.length; f++) {
            if (slotsPerFloor[f] <= 0) throw new
IllegalArgumentException("Each floor needs slots.");
            this.slots[f] = new String[slotsPerFloor[f]];
        }
        this.pricingRule = Objects.requireNonNull(pricingRule,
"pricingRule");
    }

    /** Returns {floor, slot}; returns null if full. */
    public synchronized int[] allocate(String registrationNumber) {
        if (registrationNumber == null ||
registrationNumber.trim().isEmpty())
            throw new IllegalArgumentException("Registration number
cannot be empty.");
        String reg =
registrationNumber.trim().toUpperCase(Locale.ROOT);
        if (activeByRegistration.containsKey(reg))
```

```
            throw new IllegalArgumentException("Vehicle is already
parked: " + reg);
        for (int f = 0; f < slots.length; f++) {
            for (int s = 0; s < slots[f].length; s++) {
                if (slots[f][s] == null) {
                    slots[f][s] = reg;
                    return new int[]{f, s};
                }
            }
        }
        return null;
    }

    public synchronized boolean release(int floor, int slot) {
        if (floor < 0 || floor >= slots.length || slot < 0 || slot >=
slots[floor].length)
            return false;
        if (slots[floor][slot] == null) return false;
        String reg = slots[floor][slot];
        slots[floor][slot] = null;
        ParkingTicket ticket = activeByRegistration.remove(reg);
        if (ticket != null)
activeTickets.remove(ticket.getTicketNumber());
        return true;
    }
```

```
    public synchronized ParkingTicket enter(Vehicle vehicle, boolean
monthlyPass) {
        Objects.requireNonNull(vehicle, "vehicle");
        if
(activeByRegistration.containsKey(vehicle.getRegistrationNumber())
)
            throw new IllegalArgumentException("Duplicate active
registration.");
        int[] position = allocate(vehicle.getRegistrationNumber());
        if (position == null) return null;
        ParkingTicket ticket = new ParkingTicket(vehicle, position[0],
position[1],
                LocalDateTime.now(), monthlyPass);
        activeTickets.put(ticket.getTicketNumber(), ticket);
        activeByRegistration.put(vehicle.getRegistrationNumber(),
ticket);
        totalTicketsIssued++;
        return ticket;
    }


    public synchronized double exit(String registrationNumber,
LocalDateTime exitTime,
                                    int chargingKWh) {
        String reg =
registrationNumber.trim().toUpperCase(Locale.ROOT);
        ParkingTicket ticket = activeByRegistration.get(reg);
```

```
        if (ticket == null) throw new IllegalArgumentException("No
active ticket for " + reg);
        if (exitTime.isBefore(ticket.getEntryTime()))
            throw new IllegalArgumentException("Exit time cannot be
before entry time.");


        long minutes = Duration.between(ticket.getEntryTime(),
exitTime).toMinutes();
        double fee = pricingRule.calculate(ticket.getVehicle(), minutes,
ticket.hasMonthlyPass());


        if (ticket.getVehicle() instanceof DeliveryVan) {
            DeliveryVan van = (DeliveryVan) ticket.getVehicle();
            int h = ticket.getEntryTime().getHour();
            if (h >= 9 && h < 17 && !ticket.hasMonthlyPass()) {
                long hours = (minutes + 59) / 60;
                fee = hours * van.rateForEntryTime(ticket.getEntryTime());
            }
        }
        if (ticket.getVehicle() instanceof Chargeable) {
            fee += ((Chargeable)
ticket.getVehicle()).chargingFee(chargingKWh);
        }
        release(ticket.getFloor(), ticket.getSlot());
        return fee;
    }
```

```
    public synchronized int getAvailableSlots() {
        int free = 0;
        for (String[] floor : slots)
            for (String slot : floor) if (slot == null) free++;
        return free;
    }

    public static int getTotalTicketsIssued() { return totalTicketsIssued;
}

    public synchronized void printStatus() {
        System.out.println("Available slots: " + getAvailableSlots());
        System.out.println("Active tickets: " + activeTickets.size());
    }
}

public class CampusSmartParkingSystem {
    public static void main(String[] args) {
        ParkingFacility facility = new ParkingFacility(new int[]{2, 1,
3});

        Vehicle bike = new TwoWheeler("TN01A1");
        Vehicle car = new Car("TN02B2");
        Vehicle ev = new ElectricCar("TN03C3");
```

```
        Vehicle van = new DeliveryVan("TN04D4");
        Vehicle ebike = new EBike("TN05E5");


        ParkingTicket bikeTicket = facility.enter(bike, false);
        ParkingTicket carTicket = facility.enter(car, false);
        ParkingTicket evTicket = facility.enter(ev, false);
        ParkingTicket vanTicket = facility.enter(van, true); // monthly pass: parking free
        ParkingTicket ebikeTicket = facility.enter(ebike, false);


        System.out.println("Bike ticket: " + (bikeTicket == null ? "FULL" : bikeTicket.getTicketNumber()));
        System.out.println("Car ticket: " + (carTicket == null ? "FULL" : carTicket.getTicketNumber()));
        System.out.println("EV ticket: " + (evTicket == null ? "FULL" : evTicket.getTicketNumber()));
        System.out.println("Van ticket: " + (vanTicket == null ? "FULL" : vanTicket.getTicketNumber()));
        System.out.println("E-bike ticket: " + (ebikeTicket == null ? "FULL" : ebikeTicket.getTicketNumber()));


        System.out.println("Electric-car equality by registration: " +
                new Car("TN03C3").equals(new ElectricCar("TN03C3")));

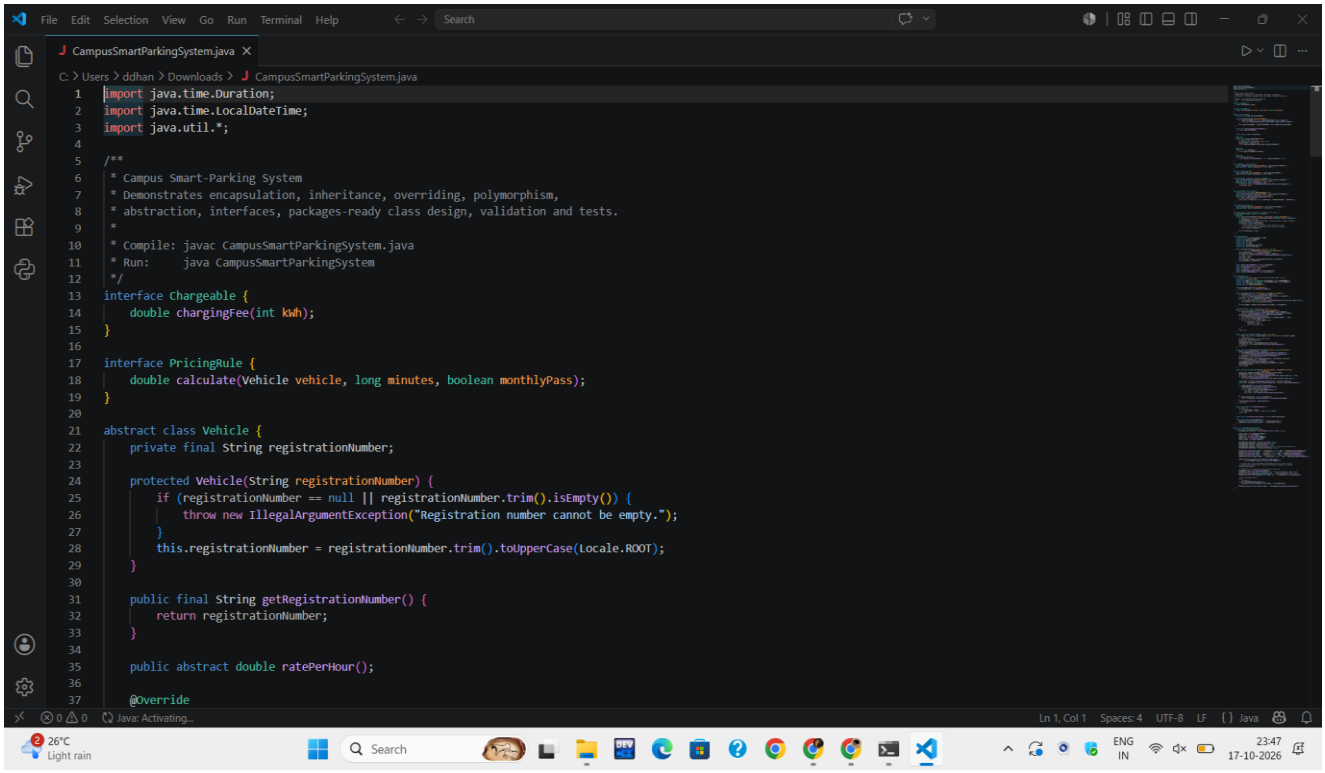

        // A one-day pass can be represented by monthlyPass=false for normal visitors;
```

```
        // extend with a PassType enum if separate one-day-pass validation is required.
        facility.printStatus();


        // Demonstrate fee calculation without waiting in real time.
        PricingRule rule = new StandardPricingRule();
        System.out.println("Car fee for 130 minutes: Rs. " + rule.calculate(car, 130, false));
        System.out.println("Monthly-pass parking fee: Rs. " + rule.calculate(van, 130, true));
        System.out.println("EV charging fee for 10 kWh: Rs. " + ((Chargeable) ev).chargingFee(10));


        // Basic validation checks
        try {
            new Car(" ");
        } catch (IllegalArgumentException ex) {
            System.out.println("Validation test passed: " + ex.getMessage());
        }
        System.out.println("Total tickets issued: " + ParkingFacility.getTotalTicketsIssued());
    }
}
```

OUTPUT:



```java
import java.time.Duration;
import java.time.LocalDateTime;
import java.util.*;

/**
 * Campus Smart-Parking System
 * Demonstrates encapsulation, inheritance, overriding, polymorphism,
 * abstraction, interfaces, packages-ready class design, validation and tests.
 *
 * Compile: javac CampusSmartParkingSystem.java
 * Run:     java CampusSmartParkingSystem
 */
interface Chargeable {
    double chargingFee(int kWh);
}

interface PricingRule {
    double calculate(Vehicle vehicle, long minutes, boolean monthlyPass);
}

abstract class Vehicle {
    private final String registrationNumber;

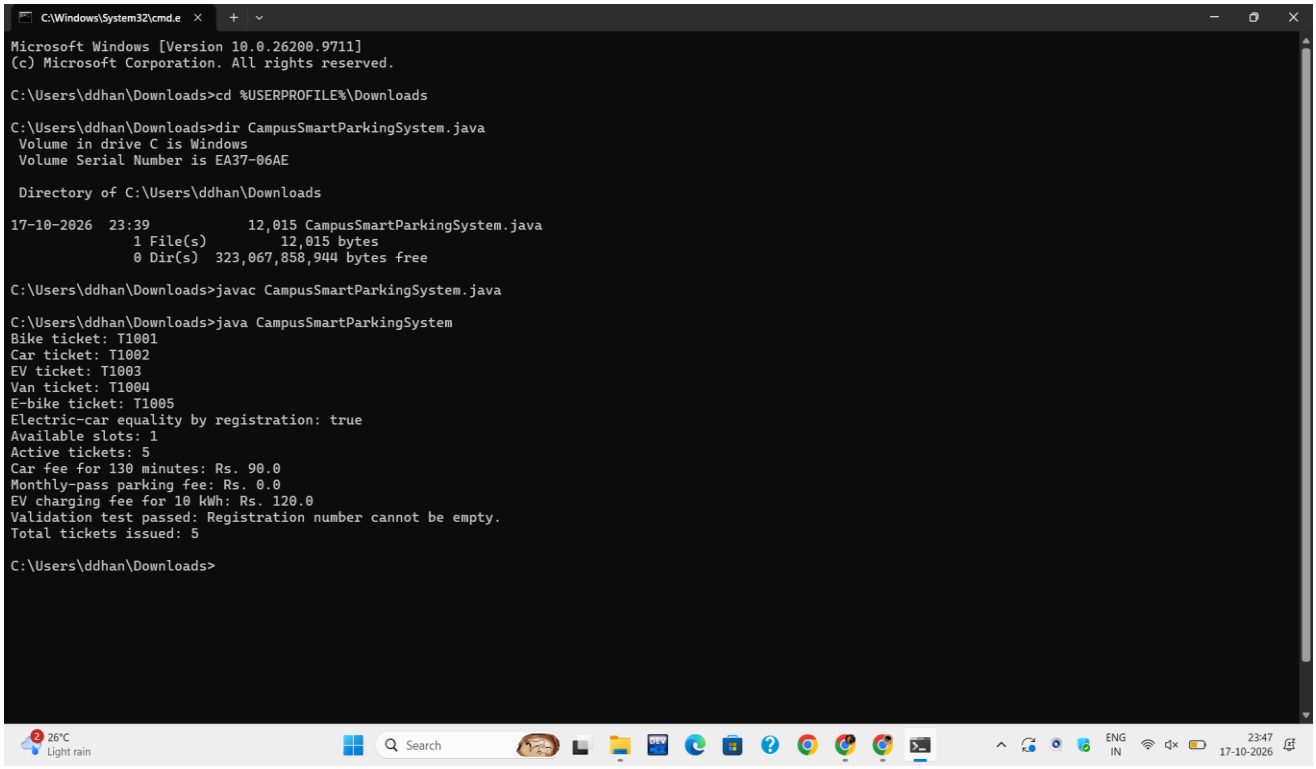
    protected Vehicle(String registrationNumber) {
        if (registrationNumber == null || registrationNumber.trim().isEmpty()) {
            throw new IllegalArgumentException("Registration number cannot be empty.");
        }
        this.registrationNumber = registrationNumber.trim().toUpperCase(Locale.ROOT);
    }

    public final String getRegistrationNumber() {
        return registrationNumber;
    }

    public abstract double ratePerHour();

    @Override
```

Java Code in Visual Studio



```
Microsoft Windows [Version 10.0.26200.9711]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ddhan\Downloads>cd %USERPROFILE%\Downloads

C:\Users\ddhan\Downloads>dir CampusSmartParkingSystem.java
 Volume in drive C is Windows
 Volume Serial Number is EA37-06AE

 Directory of C:\Users\ddhan\Downloads

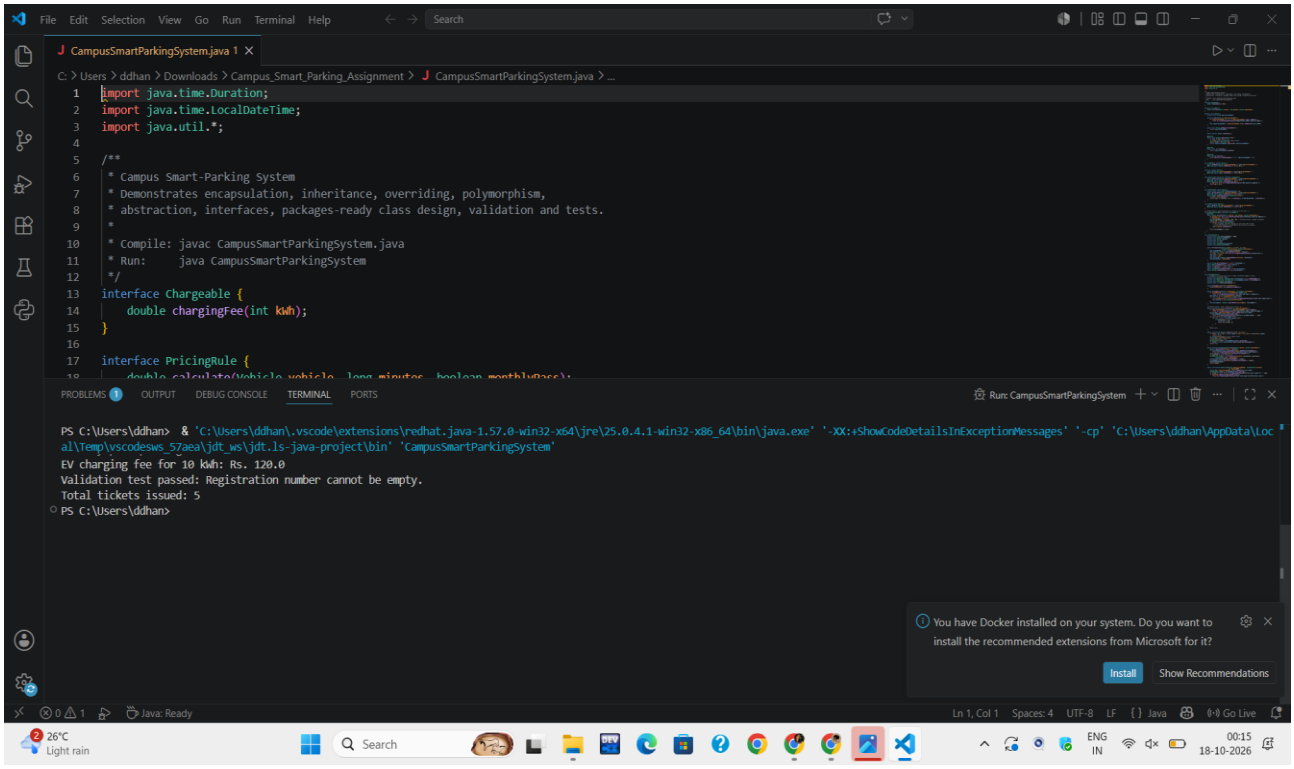
17-10-2026  23:39            12,015 CampusSmartParkingSystem.java
               1 File(s)         12,015 bytes
               0 Dir(s)  323,067,858,944 bytes free

C:\Users\ddhan\Downloads>javac CampusSmartParkingSystem.java

C:\Users\ddhan\Downloads>java CampusSmartParkingSystem
Bike ticket: T1001
Car ticket: T1002
EV ticket: T1003
Van ticket: T1004
E-bike ticket: T1005
Electric-car equality by registration: true
Available slots: 1
Active tickets: 5
Car fee for 130 minutes: Rs. 90.0
Monthly-pass parking fee: Rs. 0.0
EV charging fee for 10 kWh: Rs. 120.0
Validation test passed: Registration number cannot be empty.
Total tickets issued: 5

C:\Users\ddhan\Downloads>
```

Output in Commands



Output in Visual Studio Code